PPO Parameters Reference (Current Implementation)
Generated on 2026-07-23.

1) Objective
$g = g_{ ext{delay}} + g_{ ext{path}}$
This is the co-design accumulated cost used by PPO transitions.
Source: coarse_scheduler.py (dynamic nodes), evaluate.py (final reporting).

2) What one Action means
At each decision node, PPO chooses one index over current legal branches.
One action corresponds to one branch and that branch carries the scheduling/path-decision effects.
Code references: ppo_training/environment.py (step/reset), ppo_training/encoding.py (encode_actions).

3) State / Action dimensions
$ ext{state_dim} = 1 + N_r (d_s + 2(R+1) + 3R + 2P + M)$
where N_r=number of robots, $R=n_ ext{resources}$, $P=n_ ext{ports}$, $M= ext{max\_route\_options}$, $d_s= ext{ROBOT\_STATE\_SCALARS}=10$.
$ ext{action_dim} = N_r ((R+1) + 2(R+1) + d_a)$
where $d_a= ext{ROBOT\_ACTION\_SCALARS}=4$.
Default EncodingConfig: $R=9, P=12$.
Source: ppo_training/encoding.py lines 47-67, 130-181.

4) Reward definition
At one environment step: $r_t = -(g_{t+1}-g_t) = -(\Delta g_t)$.
Code: ppo_training/environment.py lines 79-107.

5) PPOConfig defaults in trainer.py
$\gamma = 1.0$
$\lambda = 0.95$
$\epsilon = 0.2$
$c_1 = ext{entropy\_coef} = 0.01$
$c_2 = ext{value\_loss\_coef} = 0.5$
$ lpha = ext{learning_rate} = 3e{-4}$
$ ext{update_epochs}=4, ext{minibatch_size}=64, ext{max_grad_norm}=0.5, ext{max_decisions_per_episode}=200$
$ ext{reward_norm_mode}=\{none,absmax\}, \epsilon_{norm}=1e{-12}$.
Source: ppo_training/trainer.py lines 25-39.

6) Command-line mapping in train.py

$ ext{--learning-rate} o$ PPOConfig.learning_rate.
$ ext{--update-epochs} o$ PPOConfig.update_epochs.
$ ext{--minibatch-size} o$ PPOConfig.minibatch_size.
$ ext{--entropy-coef} o$ PPOConfig.entropy_coef.
$ ext{--max-decisions} o$ PPOConfig.max_decisions_per_episode.
$ ext{--reward-norm-mode/--lr-schedule/--entropy-schedule}$ map to training behavior and schedules.
Also controls: --skip-trivial-cases, --fix-shortest-paths, --bc-pretrain-episodes.

Source: ppo_training/train.py 25-135, 337-375, 500-540.

7) GAE and targets
Temporal-difference: $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$.
Generalized Advantage: $\hat A_t = \delta_t + \gamma\lambda\hat A_{t+1}$.
Value target: $V_t^{ ext{target}} = \hat A_t + V(s_t)$.
Normalization: $ ilde A_t = (\hat A_t - \mu(\hat A))/ (\sigma(\hat A)+1e{-8})$.
Source: ppo_training/rollout_buffer.py 47-63.

8) PPO update formulas
Density ratio: $ ho_t = \exp(\log \pi_{ heta}(a_t|s_t)-\log \pi_{ heta_{old}}(a_t|s_t))$.
$L_t^{ ext{surrogate}} = \min(ho_t \hat A_t, ext{clip}(ho_t,1-\epsilon,1+\epsilon)\hat A_t)$.
$L^{ ext{actor}} = -\mathbb E[L_t^{ ext{surrogate}}]$.
$J^{ ext{actor}} = L^{ ext{actor}} - c_1\,H(\pi(\cdot|s_t))$.
$L^{ ext{critic}} = \mathbb E[(V_\phi(s_t)-V_t^{ ext{target}})^2]$.
$J^{ ext{critic}} = c_2 L^{ ext{critic}}$.
Also logged: $ ext{approx_kl}=\mathbb E[(ho_t-1)-\log ho_t],\ ext{clip_fraction}=\mathbb E[| ho_t-1|>\epsilon]$.
Source: ppo_training/trainer.py 329-360.

9) Networks
Actor: BranchScoringActor
Input is one state concatenated with each branch action vector. Shared MLP, output one logit per branch.
Critic: StateValueCritic
Input is state, output scalar $V(s)$.
Defaults: 2 hidden layers, hidden_dim=128.
Source: ppo_training/networks.py 15-97.

10) Training loop
Data flow: collect_rollouts o collect_episode o RolloutBuffer o update_with_entropy.
Within one episode: state/action encode -> actor sample/greedy -> env step -> reward/GAE storage.
Per update: shuffle all rollout steps, run $ ext{update_epochs}$ passes, minibatch updates for actor + critic.
Source: ppo_training/trainer.py 229-375.

11) Evaluation and gaps
Greedy inference gives $J_{ ext{PPO}}= ext{ppo\_cost}$; exact gives $J^*= ext{exact\_cost}$.
Absolute gap: $\Delta_{abs}=J_{ ext{PPO}}-J^*$.
Relative gap: $\Delta_{rel}= rac{\Delta_{abs}}{|J^*|}$.
If $J^*=0$, define $\Delta_{rel}=0$ if $J_{ ext{PPO}}=0$, else $\Delta_{rel}=\infty$.
Source: ppo_training/evaluate.py 109-114.

12) Exact expansion/pruning note
Environment uses immediate branch expansion without branch-and-bound pruning during rollout actions.

Exact solver pruning remains in search functions and is not part of PPO action logits directly.
Source: coarse_scheduler.py 1241-1253, 1256-1262, 1293/1319.

If you want a full LaTeX-only reference with theorem-style formatting, I can convert this to a separate file (e.g., .tex or .md) in the next step.